CO3-AT1-Assignment

View Only

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5 ✓
Manufacturing Plant – Producer-Consumer Problem

Q5 Manufacturing Plant – Producer-Consumer Problem

10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:

- Make the producer wait when the buffer is full.
- Make the consumer wait when the buffer is empty.
- Prevent duplicate consumption.
- Ensure that products are consumed only after production.
- Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution

Consumer starts first Consumer waits until a product is available Concurrency / design

10 marks

5/5 answered

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```
class ManufacuringPlant{
    static class Buffer{
        String product;
        boolean available = false;

        synchronized void produce(String p)throws
            InterruptedException{
            while(available){
                wait();
            }
            product = p;
            available = true;

            System.out.println("Produced:" +product
            notify();
        }
        synchronized void consume()throws
      InterruptedException{
            while(!available){
                wait();
            }
            System.out.println("consumed:" +product

            product = null;
            available = false;

            notify();
```

Input

stdin input

Output

```
Produced:P1
consumed:P1
Produced:P2
consumed:P2
Produced:P3
consumed:P3
```

```
28
29          }
30      }
31      static class Producer extends Thread{
32          Buffer buffer;
33
34          Producer(Buffer buffer){
35              this.buffer = buffer;
36          }
37          public void run(){
38              try{
39                  buffer.produce("P1");
40                  buffer.produce("P2");
41                  buffer.produce("P3");
42              }catch(InterruptedException e){
43                  Thread.currentThread().interrupt();
44              }
45          }
46      }
47      static class Consumer extends Thread{
48          Buffer buffer;
49
50          Consumer(Buffer buffer){
51              this.buffer = buffer;
52          }
53          public void run(){
54              try{
55                  buffer.consume();
56                  buffer.consume();
57                  buffer.consume();
58
59              }catch(InterruptedException e){
60                  Thread.currentThread().interrupt();
```

```
61              }
62          }
63      }
64      public static void main(String[]args)throws
65          Exception{
66          Buffer buffer = new Buffer();
67
68          Producer producer = new Producer(buffer);
69          Consumer consume = new Consumer(buffer);
70
71          producer.start();
72          consume.start();
73
74          producer.join();
75          consume.join();
76      }
77  }
```

Submitted